第4章　中断系统

4.1　概述

4.2　MCS-51中断系统

4.3　中断的控制

4.4　外部中断

4.5　多中断源系统设计

4.6　中断设计举例

4.1 概述

4.1.1 什么叫中断?为什么要采用中断?

中断是指计算机在执行主程序时，由于计算机系统以外的原因，暂停主程序而转去执行中断源要求服务程序，处理完毕后又返回主程序的这样一个过程。

利用中断技术可以使计算机具有更多的功能：

① 可实现高速CPU与慢速外围设备之间的配合，提高CPU的效率。

② 可实现实时处理，各控制参数可随时向CPU发出中断申请，CPU 可作出快速响应、及时处理。

③ 可实现故障的紧急处理。

④ 便于人机联系，操作人员可用键盘、开关等实现人机联系，完成人的干预控制。

4.2 MCS-51中断系统

4.2.1 中断处理过程

中断请求、 关中断、 保存断点、保护现场、开中断、 执行中断子程序、关中断、恢复现场、开中断、 中断返回

CPU在每个机器周期顺序检查每个中断源一次，(对于6MHZ晶振，则为每2μs检查一次) 发现有中断请求，则:

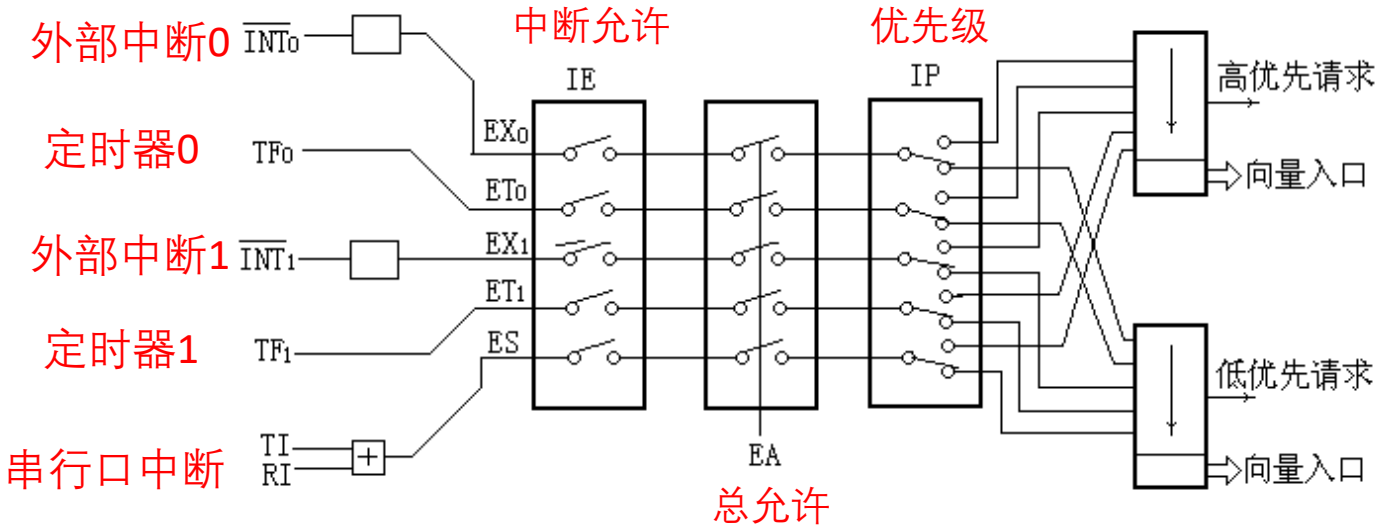
① 先响应最高优先级中断源的中断请求。

② 将发现中断信号时正在执行的那条指令执行完以后再响应中断。

③ 如发现中断信号时，正在执行的是与中断有关的指令，则需要再运行一条指令后才响应中断。

例如RETI， 或是访问IE、IP的指令

MCS-51允许5个中断请求源，提供二个中断优先级，可实现二级中断服务程序嵌套。



与中断系统相关的特殊功能寄存器有：

- **中断源寄存器：TCON（88H），SCON（98H）**
- **中断允许控制寄存器　　IE（A8H）**
- **中断优先级控制寄存器　　IP（B8H）**

4.2.2 8051的中断源

（1）5个中断源：串行口　T1　$\overline{INT1}$　T0　$\overline{INT0}$

（2）中断控制寄存器　TCON（88H）

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

其中：

ITx：外部中断触发方式选择，0：低电平触发，1：负跳变触发

IEx：外部中断请求标志，0：无中断请求，1：有中断请求

TFx：定时器/计数器溢出标志，0：定时器无溢出，1：有溢出

TRx：启、停止定时器/计数器，0：停止，1：启动

注：串行口的中断标志在SCON中锁存。

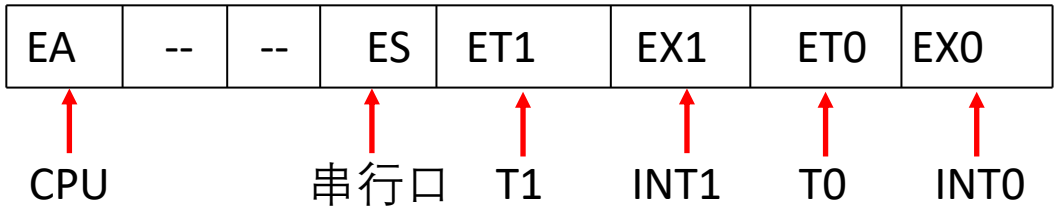
4.3 中断的控制

本节要解决如下两个问题：

(1) CPU如何控制允许中断源的中断申请?

(2) 两个以上中断源同时发来中断申请，先响应哪一个?

4.3.1 中断允许寄存器 IE 地址：(A8H) 可位寻址



相应位为1时，表示对应的中断请求可以得到响应，其中EA为总中断允许，

中断响应为两级控制-----EA和各自控制

4.3.2 中断允许设置

对相应的位置1或者清除来允许或者禁止

例： SETB EA ;开放CPU中断，可接受中断申请

SETB EX0 ;允许INT0中断

CLR ES ;禁止串行口中断

注:① 复位时IE清零，即禁止所有中断，如果某个中断源需要中断，程序中必有允许该中断源中断的语句，用SETB指令置位IE中相应的位。

② 必须逐级开放中断。总开关和子开关.

③ IE中相应位置0，该位对应的中断源被屏蔽，禁止其中断。

4.3.3 8031的中断优先级　　2级优先级

相同优先级时顺序为：外部0，定时0，外部1，定时1，串行口

中断优先级寄存器IP控制　　IP（B8H）可位寻址

			PS	PT1	PX1	PT0	PX0
			串行口	T1	INT1	T0	INT0

IP中某位置1时，该位对应的中断源被定义为高优先级中断，相反为0时，则是低优先级。

例：　SETB PT0 ；PT0置1，定时器0为高优先级中断

若：IP 值为：

			0	0	0	1	0

时，其优先级顺序为：T0、 INT0、 INT1、 T1 、串行口

注：

① 高优先级中断源的中断信号可中断任何正在执行的程序，包括主程序或其它中断源的中断服务程序；

② 同优先级中断同时发生中断请求时，内部查询序列决定了它们的优先级顺序；

③ 从高到低为：INT0 ， T0 ， INT1 ， T1，串行口。

中断响应的具体操作过程:

（1）首先将TCON中锁存的该中断源的中断标志清0。(SCON中的串行口发送接收中断标志RI、TI不能自动清0，只能用软件清0)

（2）将程序计数器PC的内容（即断点地址）压入堆栈保存，然后将PC的内容修改为该中断源的中断入口地址。

各中断源的中断入口地址固定为：

INT0：0003H　　T0：000BH　INT1：0013H

T1：001BH　串行口：0023H

分配给每个中断服务程序8个字节单元

（3）在运行中断服务程序中，RETI指令标志中断服务程序结束，要从中断返回原断点。

返回时先将原来压入堆栈保存的断点地址弹回PC，CPU从原来中断之处继续执行。

注:

1) 响应中断过程中的栈操作均由计算机自动完成，注意必须在程序中建立堆栈。

2) 在中断服务程序的开始应有保护现场的指令。

将中断服务程序中用到的各寄存器的内容压入堆栈保存，在中断结束之前弹出恢复，然后返回。这样就能保证返回原断点时各寄存器内容与中断服务前相同，使原程序能正确运行。

3) 中断与调用子程序的异同

中断过程中由主程序转向中断服务程序的请求是由外部设备提出的，发生中断的时刻不确定，而在何处调用子程序则是由程序确定的；中断和调用子程序均有栈操作。

4.4 外部中断

4.4.1 对申请中断的信号有何要求

8051 CPU 在每个机器周期采样一次INT0，INT1 脚。

（1）边沿触发（跳变触发方式）

TCON 中有外部中断的触发方式选择位和中断标志：

IE1 IT1 IE0 IT0

TCON中ITx为1时，选择边沿触发，CPU在连续两个机周期采样INTx时，前一个周期为高电平，后一个周期为低电平，则判断该负跳变触发中断，则置位中断标志IEx，申请中断。

当CPU响应中断请求后，硬件自动将IEx清0，并转入其中断入口。

如果暂时不能响应中断请求，则锁存IEx，防止中断标志丢失。

这样，边沿触发方式对申请中断信号的要求是：

要求中断请求信号至少保持高电平一个机器周期、低电平一个机器周期。

注意：当产生一次中断申请负跳变信号后，如果得到CPU响应中断后，中断标志会自动清零，因此不会产生多次中断动作。

（2） 电平触发

当TCON中IT0或IT1为0时，选择低电平触发，此时只要CPU采样到INT0或INT1脚为低电平则会触发中断。

注意：

低电平触发没有专门的中断申请标志，一旦CPU响应中断后,也不能自动清除中断申请信号。

如果CPU响应中断，在中断服务程序结束返回后，INT0或INT1脚上的申请中断的低电平信号仍未撤销，则会再次引起中断。

因此，低电平触发适用于外部中断低电平输入，且中断服务程序中能清除外部请求源的情况 。

同时，低电平触发中断的信号应当具有如下要求：

1) 请求中断的低电平信号必须保持足够长的时间，一直到中断被响应为止。否则会丢失中断信号，尤其在多级中断的情况。

2) 请求中断的低电平信号也不能太长，应保证在中断服务程序返回之前撤除该中断信号，否则会产生多余的中断动作。

3) 为了保证能被正确采样，INT端口中断请求有效信号（低电平）至少应保持2个机器周期。

4.4.2 中断申请 到 被响应的时间

CPU采样确认中断申请有效需一个机器周期，然后执行一个硬件子程序转入中断服务程序，这个硬件子程序本身占用二个机器周期，因此从产生中断请求到开始运行中断服务程序的第一条指令至少需要3个机器周期。

这里说的是至少3个机器周期，但是在以下情况下，需要延长相应时间，具体如下：

1) 发生中断信号时CPU正在执行某条指令，则要运行完该指令后才响应。这就要依该指令周期的长短增加2-3个机器周期；

2) 发生中断信号时CPU正在执行的是与中断有关的指令，则要在该指令后再运行一条指令才响应；

3) 发生中断信号时，CPU正在处理同级或更高级中断，则要那个中断服务运行完中断返回后才响应，时间不定。

总之：一个单级的中断系统，排除第3种情况外，通常响应的时间**在3-8个机器周期**之间。

4.5 多中断源系统设计

当两个外中断源不够用时，可采取不同方法增加外中断源。

4.5.1 查询法

通过多中断源并联输入，之后逐个查询的方法。

4.5.2 利用外部计数器方法

利用计数器能够对外部事件计数的功能，将计数器设置为计数一次就溢出产生中断的方法。

4.5.3 中断的软件设计

（1） 设计举例

某一系统使用中断源INT0、 T1、 串行口，并设T1为高优先级，请写出有关中断的必要指令。

```
SETB    EA          ；允许CPU接受中断
SETB    EX0         ；允许INT0中断
SETB    ET1         ；允许T1中断
SETB    ES          ；允许串行口中断
SETB    PT1         ；设T1位高优先级
```

以下指令与上述5条指令等同

```
MOV     IE，#99H    10011001
MOV     IP ，#08H   00001000
```

（2）51单片机中断程序设置

```
ORG  0000H
LJMP START
ORG 0003H
LJMP  WB0
ORG 000BH
LJMP DS0
ORG 0013H
LJMP WB1
ORG 001BH
```

```
      LJMP DS1
      ORG 0023H
      LJMP CK

      ORG 2000H
START:  .....
        .....
        SJMP $
WB0:    .....
        RETI
DS0:    .....
```

```
        RETI
WB1:    ....
        RETI
DS1:    ....
        RETI
CK:     .....
        RETI
```

说明：

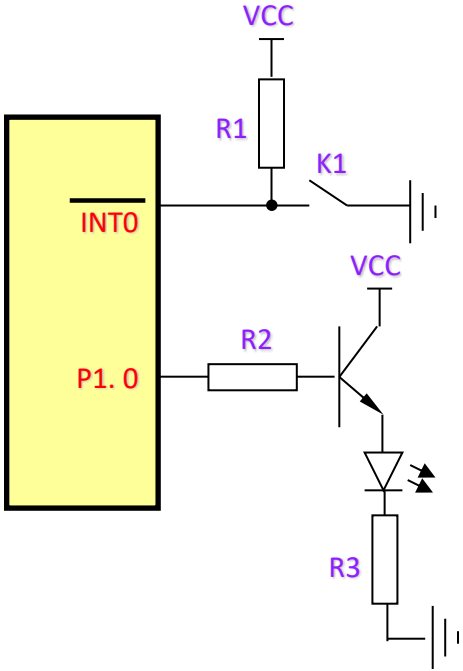
① 由于每个中断源分配的中断子程序空间只有8个字节，一般中断程序所需空间远远大于8个字节，为了不影响各中断子程序的正常执行，通常在中断入口地址单元存放一条长转移指令LJMP，使中断服务程序可在64K的存储空间内任意安排，如上述设置程序；

② 对于没有设置中断子程序的中断源，最好也进行上述设置，防止干扰引起程序“飞车”；

③ 中断子程序要尽量精简（少），提高处理器或者系统的效率和实时性。

4.6 中断设计举例

例：如图所示：8051单片机$\overline{\text{INT0}}$引脚上接有一个按键，每按一次，产生一次外部0中断，在中断程序中实现发光二级管交替亮、灭功能，并将按键次数统计于R0中，不考虑溢出。



```
        ORG 0000H
        LJMP START
        ORG 0003H
        LJMP WB0
        ……
        ORG 0080H
START:  SETB P1.0
        MOV R0,  #00H
        SETB IT0;   跳变触发方式
        SETB  EA
        SETB  EX0
HERE:   SJMP HERE
```

```
WB0:
        INC  R0
        CPL  P1.0

        RETI
        END
```

上例结果

- 未按键时， P1.0=1，LED灯 亮
- 按第1次时，P1.0=0，LED灯 灭
- 按第2次时，P1.0=1，LED灯 亮
- 按第3次时，P1.0=0，LED灯 灭
- ……

中断 小结

中断处理过程

中断处理过程分为三个阶段：中断响应、中断处理和中断返回。

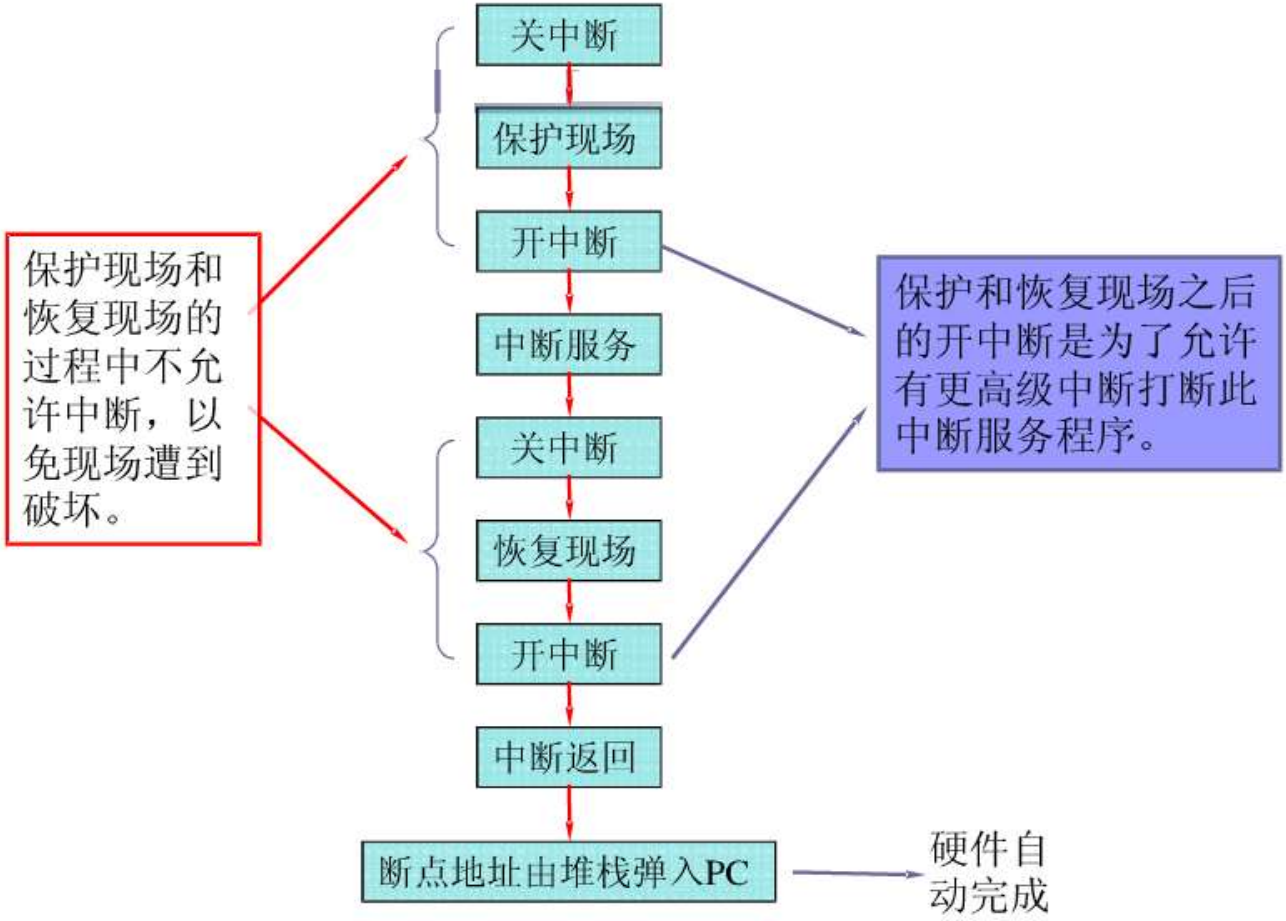
一、中断响应

二、中断处理(又称中断服务)

三、中断返回

四、中断请求的撤除

中断服务程序


关中断
保护现场
开中断
中断服务
关中断
恢复现场
开中断
中断返回
断点地址由堆栈弹入PC
硬件自动完成
保护现场和恢复现场的过程中不允许中断，以免现场遭到破坏。
保护和恢复现场之后的开中断是为了允许有更高级中断打断此中断服务程序。

复习要点

- 中断含义
- 中断向量地址
- 查询中断和向量中断
- 中断嵌套
- 外部中断触发方式
- 中断源TCON：设置中断源触发方式
- 中断允许（控制）IE：设置是否允许中断
- 中断优先级IP：设置中断优先级
- 中断初始化程序段的写法
- RET和RETI的区别